

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

CCHIRANTH B S (1BM24CS077)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING**

**in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)**

BENGALURU-560019

February 2026 to June 2026

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by CCHIRANTH B S(1BM24CS077), who is a bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Mrs Pallavi C V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index

Sl. No.	Experiment Title	Page No.
1	Write a program to obtain the Topological ordering of vertices in a given digraph.	1 - 4
2	Implement Johnson Trotter algorithm to generate permutations.	5 - 6
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	7 - 9
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	10 - 12
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	13 - 14
6	Implement 0/1 Knapsack problem using dynamic programming.	15 - 18
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	19 - 21
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	22 - 23 24 - 25
9	Implement Fractional Knapsack using Greedy technique.	26 - 27
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	28 - 29
11	Implement "N-Queens Problem" using Backtracking.	30 - 31

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program : LEETCODE - Container with most water

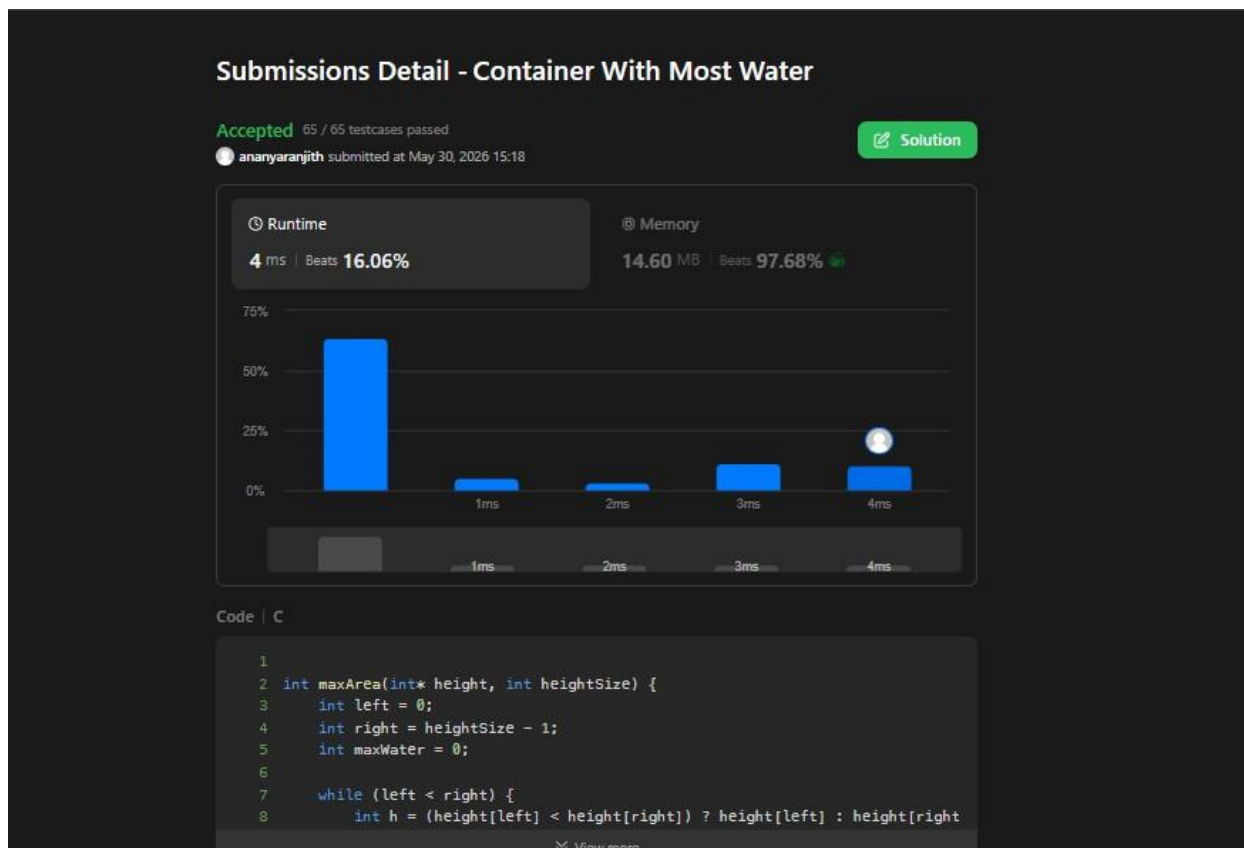
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the i^{th} line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store*.

```
int maxArea(int* height, int heightSize) {
    int left = 0;
    int right = heightSize - 1;
    int maxWater = 0;
    while (left < right) {
        int h = (height[left] < height[right]) ? height[left] :
height[right];
        int width = right - left;
        int area = h * width;
        if (area > maxWater)
            maxWater = area;
        if (height[left] < height[right])
            left++;
        else
            right--;
    }
    return maxWater;
}
```

Output:



Lab program : LEETCODE - First and Last position of element in a sorted array

Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value.

If the target is not found in the array, return [-1, -1].

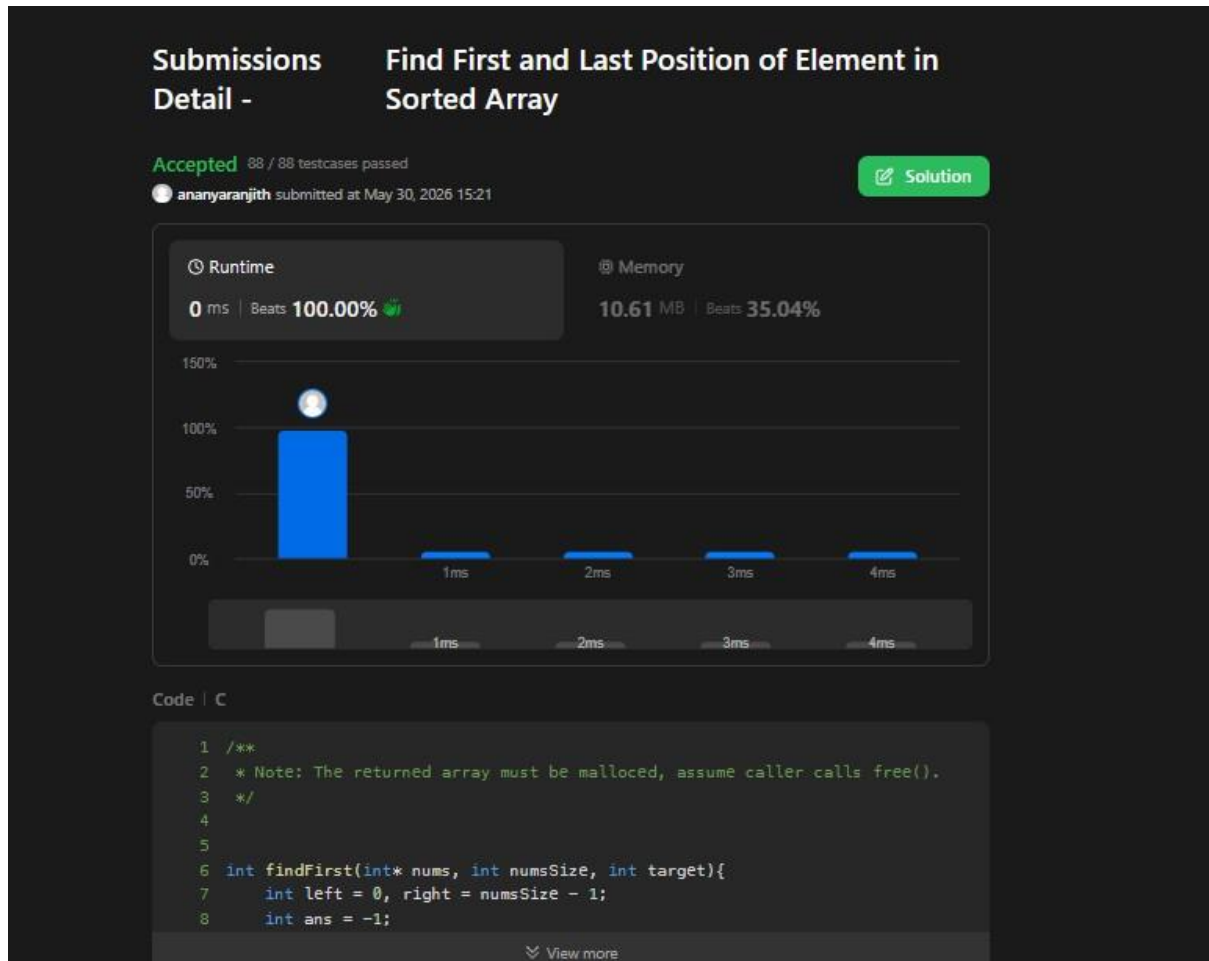
```
int findFirst(int* nums, int numsSize, int target){
    int left = 0, right = numsSize - 1;
    int ans = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            ans = mid;
            right = mid - 1;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return ans;
}

int findLast(int* nums, int numsSize, int target) {
    int left = 0, right = numsSize - 1;
    int ans = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            ans = mid;
            left = mid + 1;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return ans;
}

int* searchRange(int* nums, int numsSize, int target, int* returnSize) {
    *returnSize = 2;
    int* result = (int*)malloc(2 * sizeof(int));
    result[0] = findFirst(nums, numsSize, target);
    result[1] = findLast(nums, numsSize, target);
}
```

```
    return result;
}
```

Output:



Lab program 1:

Write a program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#define MAX 100
int main() {
    int n, graph[MAX][MAX];
    int indegree[MAX] = {0};
    int visited[MAX] = {0};
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (graph[i][j] == 1)
                indegree[j]++;
        }
    }
    printf("Topological Order: ");
    for (int count = 0; count < n; count++) {
        int found = -1;
        for (int i = 0; i < n; i++) {
            if (indegree[i] == 0 && visited[i] == 0) {
                found = i;
                break;
            }
        }
        if (found == -1) {
            printf("\nGraph has a cycle! No topological sort possible.\n");
            return 0;
        }
        printf("%d ", found);
        visited[found] = 1;
        for (int j = 0; j < n; j++) {
            if (graph[found][j] == 1)
                indegree[j]--;
        }
    }
}
```



```

    }
    printf("\n");
    return 0;
}

```

Output:

```

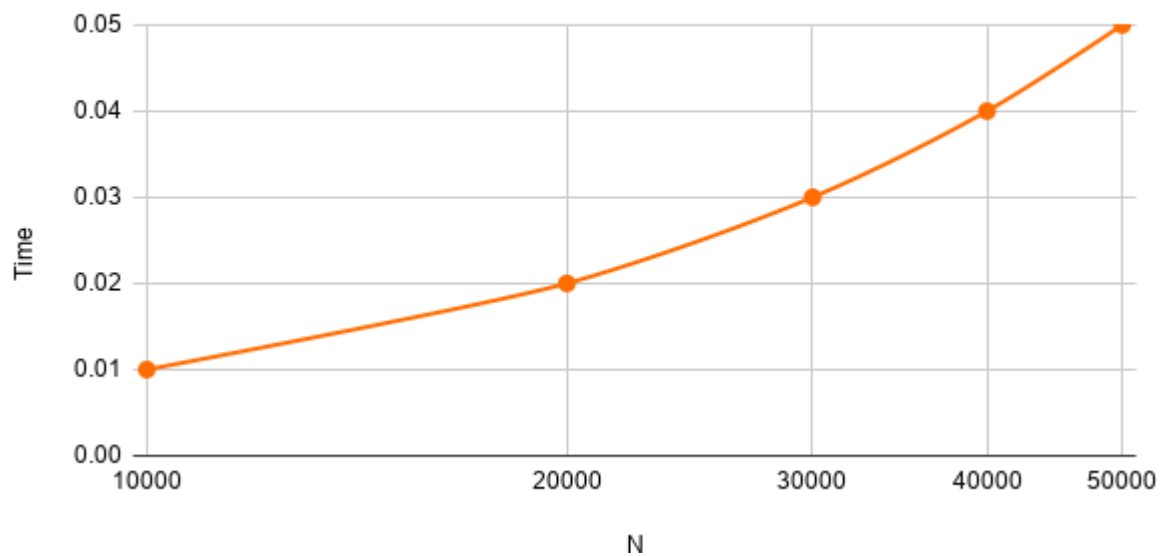
PS C:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41\" ; if ($?) { gcc topological_sort.c -o topological_sort } ; if (
$?) { .\topological_sort }
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
0 0 0 1 0
0 0 0 1 0
0 0 0 0 1
0 0 0 0 0
Topological Order: 0 1 2 3 4
PS C:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41>

```

Graph:

Topological Sort

Best Case - $\Omega(V + E)$, Average Case - $\Theta(V + E)$, Worst Case - $O(V + E)$



Lab program 4: LEETCODE - Course Schedule

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you **must** take course b_i first if you want to take course a_i.

For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize,
int* prerequisitesColSize) {
    int* indegree = (int*)calloc(numCourses, sizeof(int));
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* graphSize = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(prerequisitesSize * sizeof(int));
    }
    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        int prereq = prerequisites[i][1];

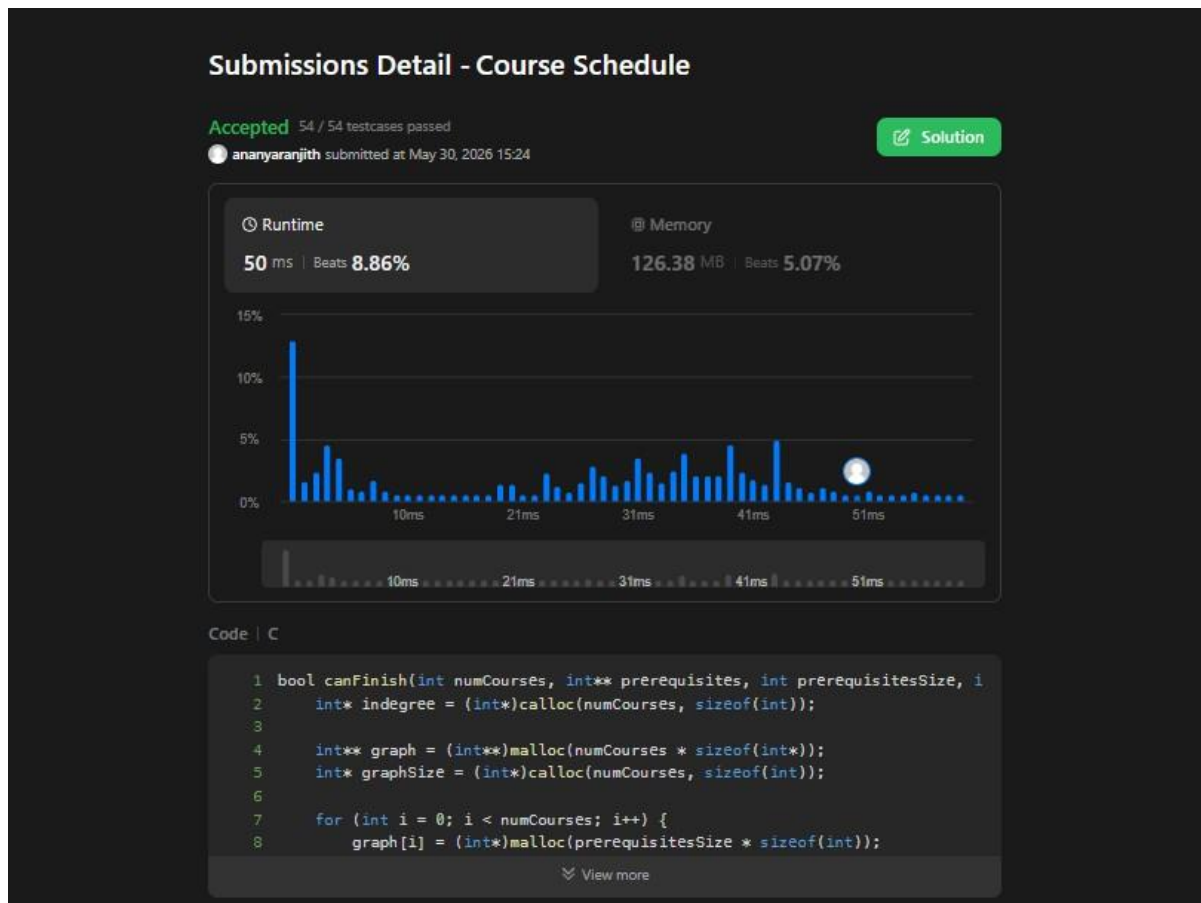
        graph[prereq][graphSize[prereq]++] = course;
        indegree[course]++;
    }
    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0)
            queue[rear++] = i;
    }
    int completed = 0;
    while (front < rear) {
        int curr = queue[front++];
        completed++;
        for (int i = 0; i < graphSize[curr]; i++) {
            int next = graph[curr][i];
            indegree[next]--;
            if (indegree[next] == 0)
                queue[rear++] = next;
        }
    }
    free(indegree);
    free(queue);
}
```

```

free(graphSize);
for (int i = 0; i < numCourses; i++)
    free(graph[i]);
free(graph);
return completed == numCourses;
}

```

Output:



Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>

int getMobile(int a[], int dir[], int n)
{
    int mobile = 0, mobile_index = -1;
    for (int i = 0; i < n; i++)
    {
        if (dir[a[i]] == -1 && i != 0 && a[i] > a[i-1] && a[i] > mobile)
        {
            mobile = a[i];
            mobile_index = i;
        }
        if (dir[a[i]] == 1 && i != n-1 && a[i] > a[i+1] && a[i] > mobile)
        {
            mobile = a[i];
            mobile_index = i;
        }
    }
    return mobile_index;
}

void printPermutation(int a[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

void johnsonTrotter(int n)
{
    int a[n], dir[n+1];
    for (int i = 0; i < n; i++)
        a[i] = i + 1;
    for (int i = 1; i <= n; i++)
        dir[i] = -1; // -1 = left, 1 = right
    printPermutation(a, n);
    while (1)
    {
        int mobile_index = getMobile(a, dir, n);
```

```

        if (mobile_index == -1)
            break;
        int mobile = a[mobile_index];
        int swap_index = mobile_index + dir[mobile];
        int temp = a[mobile_index];
        a[mobile_index] = a[swap_index];
        a[swap_index] = temp;
        mobile_index = swap_index;
        for (int i = 0; i < n; i++)
        {
            if (a[i] > mobile)
                dir[a[i]] = -dir[a[i]];
        }
        printPermutation(a, n);
    }
}

int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    johnsonTrotter(n);
    return 0;
}

```

Output:



```

PS C:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41\" ; if ($?) { gcc JOHNSON_AND_TROTTER.c -o JOHNSON_AND_TROTTER }
; if ($?) { .\JOHNSON_AND_TROTTER }
Enter number of elements: 4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4
PS C:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41>

```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>
void merge(int arr[], int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for(int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for(int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while(i < n1 && j < n2)
    {
        if(L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while(i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }
    while(j < n2)
    {
        arr[k] = R[j];
        j++;
    }
}
```

```

        k++;
    }
}

void mergeSort(int arr[], int left, int right)
{
    if(left < right)
    {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

void printArray(int arr[], int size)
{
    for(int i = 0; i < size; i++)
        printf("%d ", arr[i]);

    printf("\n");
}

int main()
{
    int arr[] = {38, 27, 43, 10};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array:\n");
    printArray(arr, n);
    mergeSort(arr, 0, n - 1);
    printf("Sorted array:\n");
    printArray(arr, n);
    return 0;
}

```

Output:

```

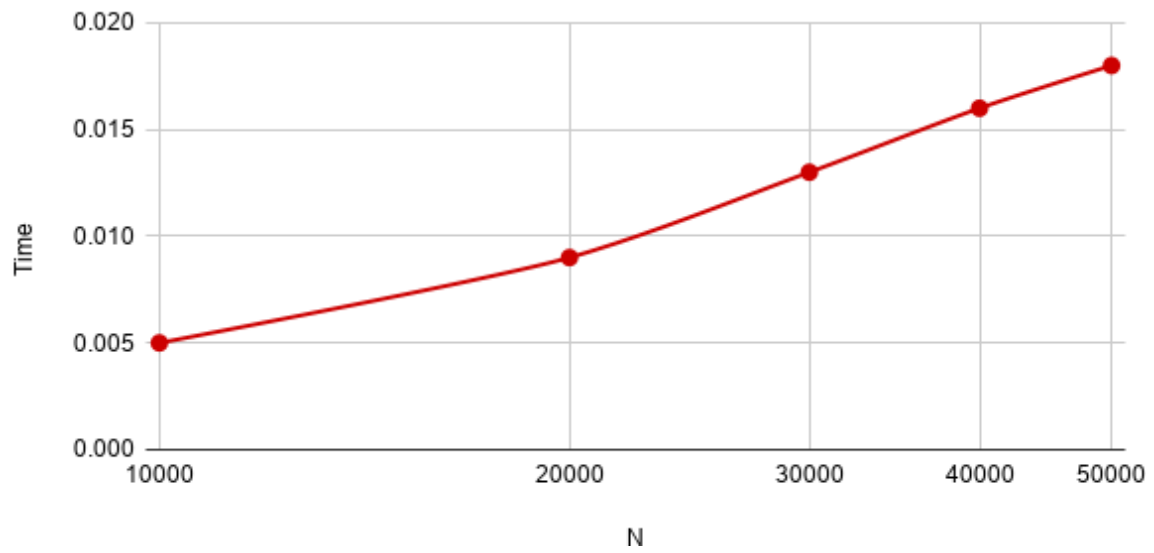
PS C:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41\" ; if ($?) { gcc merge_sort.c -o merge_sort } ; if ($?) { .\merge_sort }
Original array:
38 27 43 10
Sorted array:
10 27 38 43
PS C:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41>

```

Graph:

Merge Sort

Best case - $\Omega(n \log n)$, Average case - $\Theta(n \log n)$, Worst case - $O(n \log n)$



Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for(int j = low; j < high; j++)
    {
        if(arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quickSort(int arr[], int low, int high)
{
    if(low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
void printArray(int arr[], int n)
{
    for(int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
}
```

```

    }
    printf("\n");
}
int main()
{
    int arr[] = {38, 27, 43, 10, 9, 82};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original Array:\n");
    printArray(arr, n);
    quickSort(arr, 0, n - 1);
    printf("Sorted Array:\n");
    printArray(arr, n);
    return 0;
}

```

Output:

```

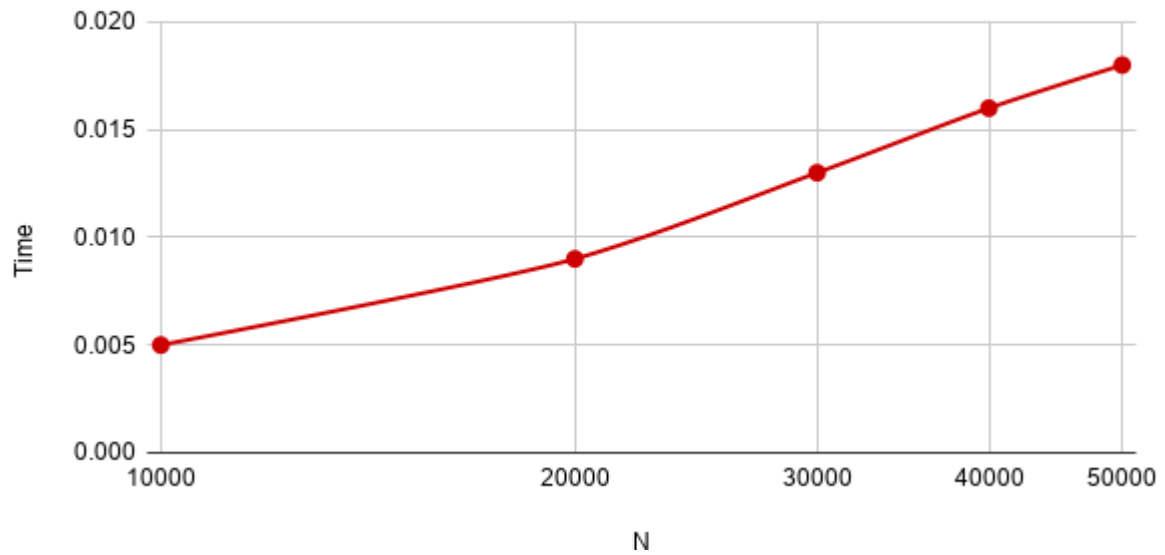
PS C:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41\" ; if ($?) { gcc quicksort.c -o quicksort } ; if ($?) { .\quicksort }
Original Array:
38 27 43 10 9 82
Sorted Array:
9 10 27 38 43 82
PS C:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41>

```

Graph:

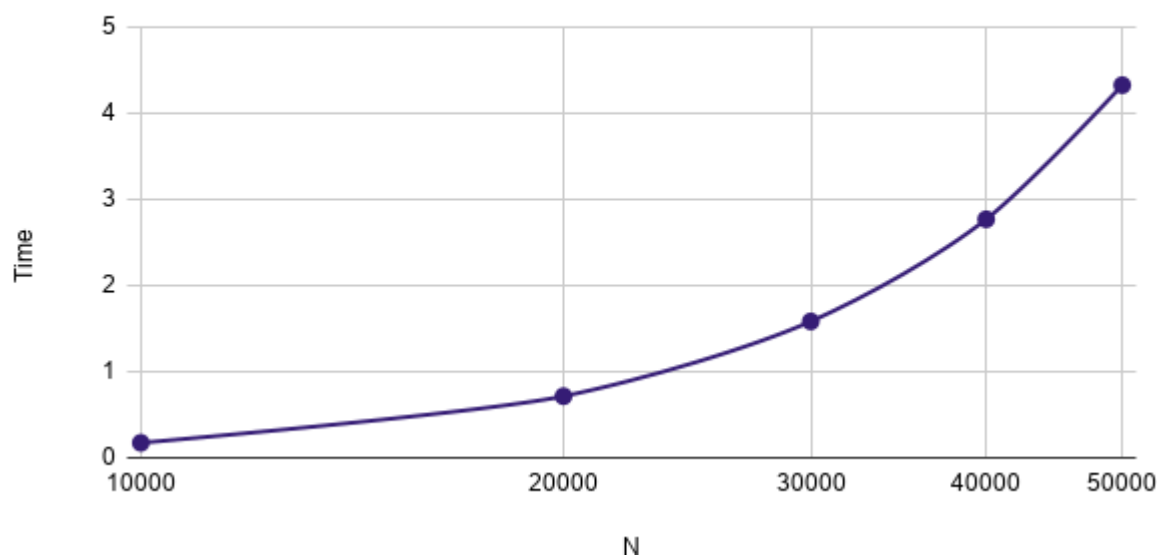
Quick Sort

Best case - $\Omega(n \log n)$, Average case - $\Theta(n \log n)$



Quick Sort

Worst case - $O(n^2)$



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;
    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main(){
    int n;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter the elements: ");
    for(int i = 0; i < n; i++){
        scanf("%d", &arr[i]);
    }
    heapSort(arr, n);
}
```

```

printf("Sorted array: ");
for(int i = 0; i < n; i++){
    printf("%d ", arr[i]);
}
printf("\n");
return 0;
}

```

Output:

```

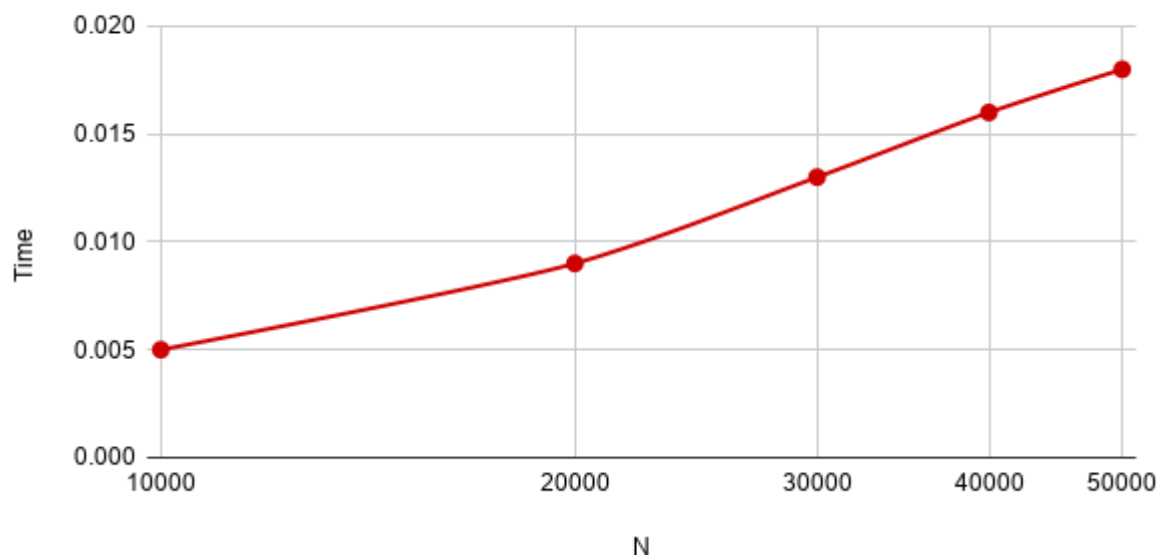
PS C:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0\" ; if ($?) { gcc heapsort.c -o heapsort } ; if ($?) { .\heapsort }
Enter the number of elements: 9
Enter the elements: 100
10
80
5
6
40
60
2
3
Sorted array: 2 3 5 6 10 40 60 80 100

```

Graph:

Heap Sort

Best case - $\Omega(n \log n)$, Average case - $\Theta(n \log n)$, Worst case - $O(n \log n)$



Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include<stdio.h>
int n,m,w[10],p[10],v[20][20];
int max(int a,int b){
    return (a>b)?a:b;
}
void knapsack(int n, int m, int w[],int p[]){
    int i,j;
    for(i=0;i<=n;i++){
        for(j=0;j<=m;j++){
            if(i==0 || j==0){
                v[i][j]=0;
            }
            else if(w[i-1] > j){
                v[i][j]=v[i-1][j];
            }
            else{
                v[i][j]=max(v[i-1][j],
                    p[i-1] + v[i-1][j - w[i-1]]);
            }
        }
    }
}
int main(){
    int i,j;
    printf("Enter number of items: ");
    scanf("%d",&n);
    printf("Enter capacity: ");
    scanf("%d",&m);
    printf("Enter weights:\n");
    for(i=0;i<n;i++)
        scanf("%d",&w[i]);
    printf("Enter profits:\n");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    knapsack(n,m,w,p);
    printf("\nDP Table:\n");
    for(i=0;i<=n;i++){
```

```

        for(j=0;j<=m;j++){
            printf("%d ",v[i][j]);
        }
        printf("\n");
    }
    printf("\nMaximum Profit = %d\n", v[n][m]);
    return 0;
}

```

Output:

```

PS C:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0\" ; if ($?) { gcc 1KNAPSACK.c -o 1KNAPSACK } ; if ($?) { .\1KNAPSACK }
Enter number of items: 4
Enter capacity: 10
Enter weights:
5
4
6
3
Enter profits:
10
40
30
50

DP Table:
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 10 10 10 10 10
0 0 0 0 40 40 40 40 40 50
0 0 0 0 40 40 40 40 40 50
0 0 0 50 50 50 90 90 90 90

Maximum Profit = 90

```

Lab program 6: LEETCODE -Minimum Swaps to make Sequences Increasing

You are given two integer arrays of the same length `nums1` and `nums2`. In one operation, you are allowed to swap `nums1[i]` with `nums2[i]`.

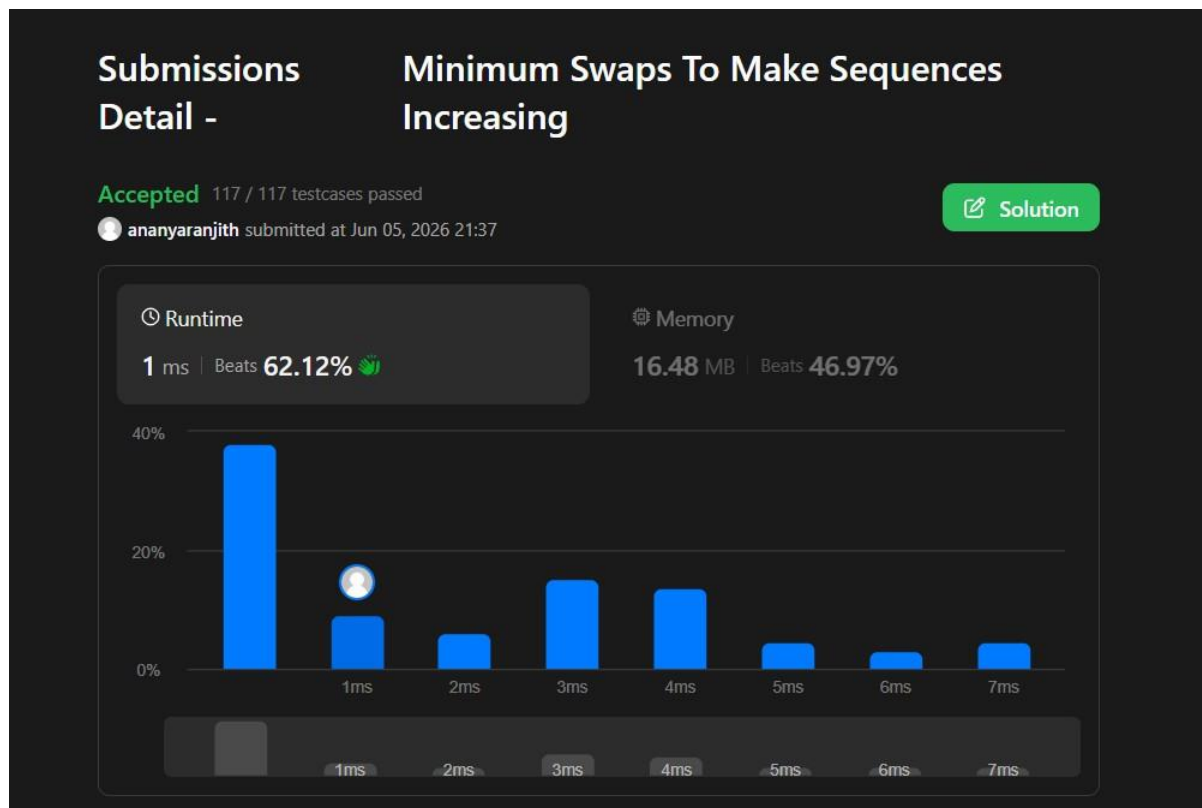
For example, if `nums1 = [1,2,3,8]`, and `nums2 = [5,6,7,4]`, you can swap the element at `i = 3` to obtain `nums1 = [1,2,3,4]` and `nums2 = [5,6,7,8]`.

Return the *minimum number of needed operations to make* `nums1` and `nums2` **strictly increasing**. The test cases are generated so that the given input always makes it possible.

An array `arr` is **strictly increasing** if and only if `arr[0] < arr[1] < arr[2] < ... < arr[arr.length - 1]`.

```
#include <limits.h>
int min(int a, int b) {
    return a < b ? a : b;
}
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int keep = 0;
    int swap = 1;
    for (int i = 1; i < nums1Size; i++) {
        int newKeep = INT_MAX;
        int newSwap = INT_MAX;
        if (nums1[i] > nums1[i - 1] &&
            nums2[i] > nums2[i - 1]) {
            newKeep = min(newKeep, keep);
            newSwap = min(newSwap, swap + 1);
        }
        if (nums1[i] > nums2[i - 1] &&
            nums2[i] > nums1[i - 1]) {
            newKeep = min(newKeep, swap);
            newSwap = min(newSwap, keep + 1);
        }
        keep = newKeep;
        swap = newSwap;
    }
    return min(keep, swap);
}
```


Output:



Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>
int a[10][10], D[10][10], n;
void floyd(int a[][10], int n);
int min(int a, int b);
int main() {
    int i, j;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the cost adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }
    floyd(a, n);
    printf("Distance Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            printf("%d ", D[i][j]);
        }
        printf("\n");
    }
    return 0;
}
void floyd(int a[][10], int n) {
    int i, j, k;
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            D[i][j] = a[i][j];
        }
    }
    for(k = 0; k < n; k++) {
        for(i = 0; i < n; i++) {
            for(j = 0; j < n; j++) {
                D[i][j] = min(D[i][j], D[i][k] + D[k][j]);
            }
        }
    }
}
```

```

    }
}

int min(int a, int b) {
    if(a < b)
        return a;
    else
        return b;
}

```

Output:

```

PS C:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0\" ; if ($?) { gcc floyd.c -o floyd } ; if ($?) { .\floyd }
Enter the number of vertices: 4
Enter the cost adjacency matrix:
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0
Distance Matrix:
0 5 8 9
999 0 3 4
999 999 0 1
999 999 999 0
PS C:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0>

```

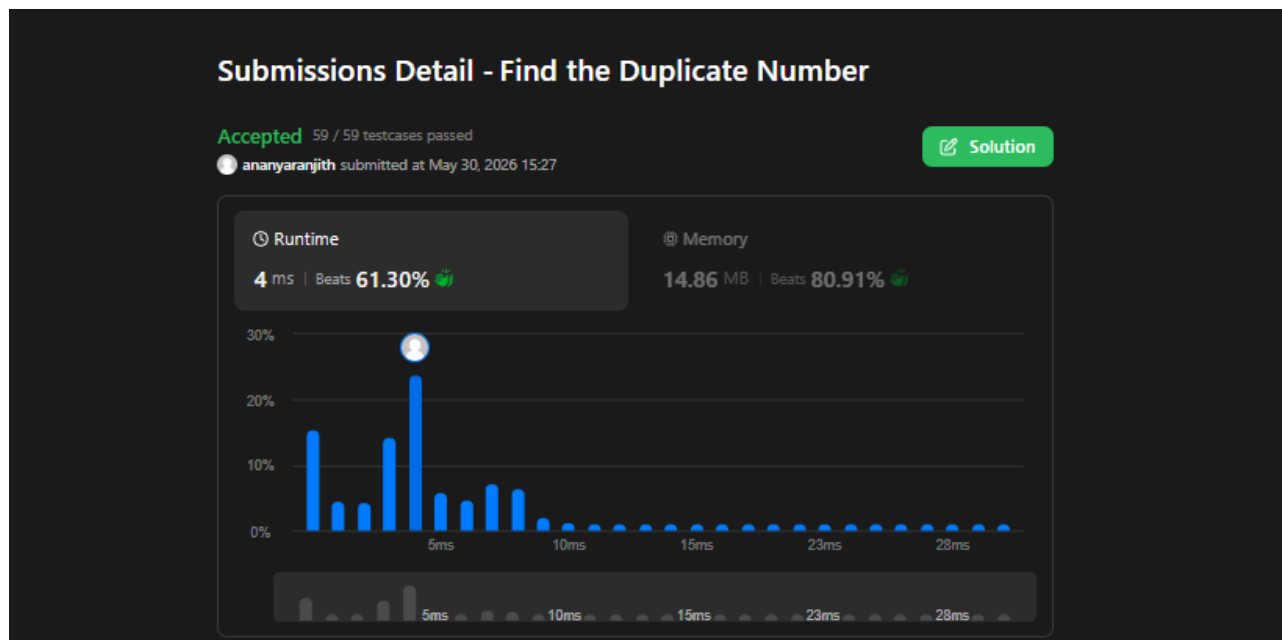
Lab program 7: LEETCODE - Find the duplicate number

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only **one repeated number** in `nums`, return *this repeated number*.

```
int findDuplicate(int* nums, int numsSize) {  
    int slow = nums[0];  
    int fast = nums[0];  
    do {  
        slow = nums[slow];  
        fast = nums[nums[fast]];  
    } while (slow != fast);  
    slow = nums[0];  
    while (slow != fast) {  
        slow = nums[slow];  
        fast = nums[fast];  
    }  
    return slow;  
}
```

Output:



Lab program 8a:

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include<stdio.h>
int cost[10][10], n, t[10][2], sum;
void prims(int cost[10][10], int n);
int main(){
    int i, j;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
    for(i=0; i<n; i++){
        for(j=0; j<n; j++){
            scanf("%d", &cost[i][j]);
        }
    }
    prims(cost, n);
    printf("Edges in the Minimum Spanning Tree:\n");
    for(i=0; i<n-1; i++){
        printf("%d %d\n", t[i][0], t[i][1]);
    }
    printf("Total cost of the Minimum Spanning Tree: %d\n", sum);
    return 0;
}

void prims(int cost[10][10], int n){
    int i, j, u, v;
    int min, source;
    int p[10], d[10], s[10];
    min=999;
    source=0;
    for(i=0; i<n; i++){
        p[i]=source;
        d[i]=cost[source][i];
        s[i]=0;
    }
    s[source]=1;
    sum=0;
    int k=0;
    for(i=0; i<n-1; i++){
        min=999;
```

```

    u=-1;
    for(j=0; j<n; j++){
        if(s[j]==0 && d[j]<min){
            min=d[j];
            u=j;
        }
    }
    if (u!=-1){
        t[k][0]=u;
        t[k][1]=p[u];
        sum+=cost[u][p[u]];
        k++;
        s[u]=1;
        for(v=0; v<n; v++){
            if(s[v]==0 && cost[u][v]<d[v]){
                p[v]=u;
                d[v]=cost[u][v];
            }
        }
    }
}
}}
```

Output:

```

PS C:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0\" ; if ($?) { gcc prim.c -o prim } ; if ($?) { .\prim }
Enter number of vertices: 4
Enter cost matrix:
0 1 5 2
1 0 99 99
5 99 0 3
2 99 3 0
Edges in the Minimum Spanning Tree:
1 0
3 0
2 3
Total cost of the Minimum Spanning Tree: 6
```

Lab program 8b:

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include<stdio.h>
int cost[10][10], n, t[10][2], sum;
void kruskal(int cost[10][10], int n);
int find(int parent[10], int i);
int main(){
    int i, j;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the cost matrix:\n");
    for(i=0; i<n; i++){
        for(j=0; j<n; j++){
            scanf("%d", &cost[i][j]);
        }
    }
    kruskal(cost, n);
    printf("Edges in the Minimum Spanning Tree:\n");
    for(i=0; i<n-1; i++){
        printf("%d %d\n", t[i][0], t[i][1]);
    }
    printf("Total cost of the Minimum Spanning Tree: %d\n", sum);
    return 0;
}
void kruskal(int cost[10][10], int n){
    int min, u, v, count, k;
    int parent[10];
    int i, j;
    k = 0;
    sum = 0;
    for(i=0; i<n; i++){
        parent[i] = i;
    }
    count = 0;
    while(count < n-1){
        min = 999;
        for(i=0; i<n; i++){
            for(j=0; j<n; j++){
                if(cost[i][j] != 0 && cost[i][j] < min){
                    min = cost[i][j];

```

```

        u = i;
        v = j;
    }
}
}
int root_u = find(parent, u);
int root_v = find(parent, v);
if(root_u != root_v){
    t[k][0] = u;
    t[k][1] = v;
    sum += min;
    parent[root_v] = root_u;
    k++;
    count++;
}
cost[u][v] = cost[v][u] = 999;
}
}
int find(int parent[10], int i){
    while(parent[i] != i){
        i = parent[i];
    }
    return i;
}

```

Output:

```

PS C:\Users\BMSCE-SCE-L3-24\Desktop\BM-41> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0\" ; if ($?) { gcc kruskal.c -o kruskal } ; if ($?) { .\kruskal }
Enter the number of vertices: 4
Enter the cost matrix:
0 1 5 2
1 0 99 99
5 99 0 3
2 99 3 0
Edges in the Minimum Spanning Tree:
0 1
0 3
2 3
Total cost of the Minimum Spanning Tree: 6

```


Lab program 9:

Implement fractional knapsack problem using Greedy technique.

```
#include<stdio.h>
struct Item{
    int weight;
    int profit;
    float ratio;
};
void swap(struct Item *a, struct Item *b){
    struct Item temp=*a;
    *a=*b;
    *b=temp;
}
void sortItems(struct Item arr[], int n){
    for(int i=0; i<n-1; i++){
        for(int j=0; j<n-i-1; j++){
            if(arr[j].ratio < arr[j+1].ratio){
                swap(&arr[j], &arr[j+1]);
            }
        }
    }
}
float fk(struct Item arr[], int n, int capacity){
    float totalProfit=0.0;
    for(int i=0; i<n; i++){
        if(capacity >= arr[i].weight){
            totalProfit += arr[i].profit;
            capacity -= arr[i].weight;
        }else{
            totalProfit += arr[i].ratio * capacity;
            break;
        }
    }
    return totalProfit;
}
int main(){
    int n, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
```

```

struct Item arr[n];
printf("Enter weight and profit of each item:\n");
for(int i=0; i<n; i++){
    scanf("%d %d", &arr[i].weight, &arr[i].profit);
    arr[i].ratio = (float)arr[i].profit / arr[i].weight;
}
printf("Enter knapsack capacity: ");
scanf("%d", &capacity);
sortItems(arr, n);
float maxProfit = fk(arr, n, capacity);
printf("Max profit = %.2f\n", maxProfit);
return 0;
}

```

Output:

```

PS C:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41\" ; if ($?) { gcc fractional_knapsack.c -o fractional_knapsack }
; if ($?) { .\fractional_knapsack }
Enter number of items: 4
Enter weight and profit of each item:
4 6
7 8
13 56
7 19
Enter knapsack capacity: 20
Max profit = 75.00
PS C:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41>

```

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>
#define INF 9999
int main() {
    int n, src;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int graph[n][n], dist[n], visited[n];
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    printf("Enter source: ");
    scanf("%d", &src);
    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }
    dist[src] = 0;
    for (int i = 0; i < n - 1; i++) {
        int min = INF, u = -1;
        for (int j = 0; j < n; j++) {
            if (!visited[j] && dist[j] < min) {
                min = dist[j];
                u = j;
            }
        }
        if (u == -1) break;
        visited[u] = 1;
        for (int j = 0; j < n; j++) {
            if (graph[u][j] && !visited[j] &&
                dist[u] + graph[u][j] < dist[j]) {
                dist[j] = dist[u] + graph[u][j];
            }
        }
    }
}
```

```

    }
    printf("Shortest distances:\n");
    for (int i = 0; i < n; i++) {
        printf("%d -> %d = %d\n", src, i, dist[i]);
    }
    return 0;
}

```

Output:

```

PS C:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\an BM24-41\" ; if ($?) { gcc dijkstras.c -o dijkstras } ; if ($?) { .\dijkstras }
Enter number of vertices: 4
Enter adjacency matrix:
0 5 0 10
0 0 3 0
0 0 0 1
0 0 0 0
Enter source: 0
Shortest distances:
0 -> 0 = 0
0 -> 1 = 5
0 -> 2 = 8
0 -> 3 = 9

```

Lab program 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 20
int board[MAX], n;
int solutionCount = 0;
int place(int row)
{
    int i;
    for(i = 1; i < row; i++)
    {
        if(board[i] == board[row] ||
           abs(board[i] - board[row]) == abs(i - row))
        {
            return 0;
        }
    }
    return 1;
}
void printSolution()
{
    int i, j;
    solutionCount++;
    printf("\nSolution %d:\n\n", solutionCount);
    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= n; j++)
        {
            if(board[i] == j)
                printf(" 1 ");
            else
                printf(" 0 ");
        }
        printf("\n");
    }
}
```

```

void queen(int row)
{
    int col;
    for(col = 1; col <= n; col++)
    {
        board[row] = col;
        if(place(row))
        {
            if(row == n)
            {
                printSolution();
            }
            else
            {
                queen(row + 1);
            }
        }
    }
}

int main()
{
    printf("Enter number of queens: ");
    scanf("%d", &n);
    queen(1);
    printf("\nTotal Possible Solutions = %d\n", solutionCount);
    return 0;
}

```

Output:

```

PS C:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0> cd "c:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0\" ; if ($?) { gcc nqueen.c -o nqueen } ; if ($?) { .\nqueen }
Enter number of queens: 4

Solution 1:

0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

Solution 2:

0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

Total Possible Solutions = 2
PS C:\Users\BMSCE-SCE-L3-24\Desktop\BM-41\0>

```

